SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 16

Name of Student : K. laxman
Enrollment No. : 2303A51428
Batch No. : 21

Task Descrption-1: (Design Database Schema for Student Management System)
Instructions:
• Using AI tools, generate CREATE TABLE statements for a Student Management Database.
• Tables: Students, Courses, Enrollments.
• Include primary keys, foreign keys, and suitable data types.
Ensure the schema follows normalization rules (1NF, 2NF, 3NF).
Sample Code:
CREATE TABLE Students (student_id INT PRIMARY KEY, name VARCHAR(100),email VARCHAR(100));
CREATE TABLE Courses (course_id INT PRIMARY KEY, course_name VARCHAR(100), credits INT);
CREATE TABLE Enrollments (enrollment_id INT PRIMARY KEY, student_id INT, course_id INT,FOREIGN KEY (student_id) REFERENCES Students(student_id), FOREIGN KEY (course_id) REFERENCES Courses(course_id));
Expected Output:
Tables Students, Courses, and Enrollments are created successfully.
Foreign key constraints are properly enforced.

Prompt:-
create a design database schema for student management system. include create table statements for students, courses, and enrollments. ensure that the schema supports many-to-many relationships between students and courses, and includes appropriate primary and foreign keys and suitable data types for each field. ensure that the schema is normalized rules (1NF, 2NF, 3NF) to eliminate redundancy and ensure data integrity. provide SQL queries to create the tables and establish the relationships between them.

SR UNIVERSITY

CODE:

```sql
-- SQLit.sql
-- create a design database schema for student management system. include create table statements for students, courses, and e
-- Create the Students table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    date_of_birth DATE NOT NULL
);

-- Create the Courses table
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
-- Create the Enrollments table to establish many-to-many relationship between Students and Courses
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id) -- Ensure a student can enroll in a course only once
);
-- The above schema is normalized to 3NF, eliminating redundancy and ensuring data integrity.
-- SQL queries to create the tables and establish relationships have been provided above.


INSERT INTO Students (first_name, last_name, email, date_of_birth) VALUES ('John', 'Doe', 'john.doe@example.com', '2000-01-01'
INSERT INTO Courses (course_name, course_code, credits) VALUES ('Introduction to Computer Science', 'CS101', 3);
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1,
1, '2024-09-01');

INSERT into Students (first_name, last_name, email, date_of_birth) VALUES ('Jane', 'Smith', 'jane.smith@example.com', '2000-02
```

```sql
insert into Students (first_name, last_name, email) values ('John', 'Doe', 'john.doe@example.com');
insert into Courses (course_name, course_code) values ('Mathematics', 'MATH101');
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 1, '2024-01-15');
show tables;
select * from Students;
select * from Courses;
select * from Enrollments;
```

Output:-

student_id	first_name	last_name	email	date_of_birth
1	John	Doe	john.doe@example.com	2000-01-01
2	Jane	Smith	jane.smith@example.com	2000-02-02

SQL ▾ 1 / 1 1 - 2 of 2

course_id	course_name	course_code	credits
1	Introduction to Computer Science	CS101	3
2	Data Structures	CS102	4

SQL ▾ 1 / 1 1 - 3 of 3

enrollment_id	student_id	course_id	enrollment_date
1	1	1	2024-09-01
2	2	1	2024-09-01
3	1	2	2024-09-02

Search

OBSERVATION:
The database is designed to manage students, courses, and their enrollments using three tables: *Students*, *Courses*, and *Enrollments*.
The *Students* table stores basic details like first name, last name, and email, with student_id acting as a unique identifier for each student. Similarly, the *Courses* table keeps information about each course, such as its name and code, with course_id as the unique key.
The *Enrollments* table connects students and courses, showing which student is enrolled in which course. It uses student_id and course_id as foreign keys and also stores the enrollment date. Since a student can take multiple courses and a course can have multiple students, this table handles that many-to-many relationship.
To avoid duplicate entries (like the same student enrolling in the same course twice), a UNIQUE constraint is added on the combination of student_id and course_id.
Overall, this design follows normalization rules, which helps keep the data organized, avoids unnecessary duplication, and ensures everything is stored in the right place.

Task Description- 2: (Insert Sample Records)
Instructions: Use AI tools to generate INSERT statements to populate the tables.
• Insert at least 3 students, 4 courses, and 5 enrollment records.
Verify data using SELECT queries.
Sample Code:
INSERT INTO Students VALUES(1, 'Arjun', 'arjun@gmail.com'), (2, 'Meera', 'meera@gmail.com'), (3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES (101, 'Database Systems', 4), (102, 'Data Structures', 3), (103, 'Operating Systems', 4), (104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES (1, 1, 101), (2, 1, 102), (3, 2, 103), (4, 3, 101), (5, 3, 104);
Expected Output:
• Sample records are inserted successfully.
• FROM Students; displays all student records.
• SELECT * FROM Enrollments; displays all enrollments.

Prompt:-
insert the values into the tables of the student management system and display the tables to verify the data insertion. insert atleast 3 students, 4 courses, and 5 enrollnment records using select queries.

SR UNIVERSITY

Code:-

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(10) UNIQUE NOT NULL
);
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)  -- To prevent duplicate enrollments
);

insert into Students (first_name, last_name, email) values ('John', 'Doe', 'john.doe@gmail.com');
insert into Students (first_name, last_name, email) values ('Jane', 'Smith', 'jane.smith@gmail.com');
insert into Students (first_name, last_name, email) values ('Bob', 'Johnson', 'bob.johnson@gmail.com');
insert into Courses (course_name, course_code) values ('Mathematics', 'MATH101');
insert into Courses (course_name, course_code) values ('Physics', 'PHYS101');
insert into Courses (course_name, course_code) values ('Chemistry', 'CHEM101');
insert into Courses (course_name, course_code) values ('Biology', 'BIO101');
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 1, '2024-01-15');
insert into Enrollments (student_id, course_id, enrollment_date) values (1, 2, '2024-01-16');
insert into Enrollments (student_id, course_id, enrollment_date) values (2, 1, '2024-01-17');
insert into Enrollments (student_id, course_id, enrollment_date) values (2, 3, '2024-01-18');
insert into Enrollments (student_id, course_id, enrollment_date) values (3, 4, '2024-01-19');

select * from students;
select * from courses;
select * from enrollments;
```

Output:-

SQL ▼ 1 / 1 1 - 3 of 3

student_id	first_name	last_name	email
NULL	John	Doe	john.doe@gmail.com
NULL	Jane	Smith	jane.smith@gmail.com
NULL	Bob	Johnson	bob.johnson@gmail.com

SQL ▼ 1 / 1 1 - 4 of 4

course_id	course_name	course_code
NULL	Mathematics	MATH101
NULL	Physics	PHYS101
NULL	Chemistry	CHEM101
NULL	Biology	BIO101

SQL ▼ 1 / 1 1 - 5 of 5

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

OBSERVATION:

The SQL code creates three tables: *Students*, *Courses*, and *Enrollments*, each with the right primary keys, foreign keys, and data types. It then adds some sample data to show how students and courses are connected through a many-to-many relationship. Finally, it displays the data from each table so you can check that everything was created and inserted correctly.

Task Description- 3: (Perform CRUD Operations)
Instructions:
Use AI to generate SQL queries for basic CRUD operations:
• Retrieve all courses
• Update a student's email
• Delete an enrollment record
• Validate the changes using SELECT statements.
Sample Code:
SELECT * FROM Courses;
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
DELETE FROM Enrollments
WHERE enrollment_id = 5;
Expected Output:
• Updated email is reflected in Students table.
• Deleted enrollment record is removed successfully.
• Remaining data remains unchanged.

Prompt:
SQL Queries for basic CRUD opertions like retrive all courses, update a student's email, delete an enrollment record, validate the changes using select statements.

Code:-

```sql
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student's email
UPDATE Students
SET email = 'hbxjhbzdhb'
WHERE student_id = 1;
-- Validate the change using SELECT statement
SELECT * FROM Students WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments
WHERE enrollment_id = 1;
-- Validate the deletion using SELECT statement
SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Output:-

SQL ▾ 1 / 1 1 - 2 of 2

course_id	course_name	course_code	credits
1	Introduction to Computer Science	CS101	3
2	Data Structures	CS102	4

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 1 of 1

student_id	first_name	last_name	email	date_of_birth
1	John	Doe	hbxjhbzdhb	2000-01-01

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

OBSERVATION:

The SQL queries show the basic CRUD operations in the student management system. The first query retrieves all courses so we can see what's available. The second query updates a student's email, helping keep their contact details accurate. The third query deletes an enrollment record, which is useful when a student drops a course. Finally, the select statements display the updated student details and remaining enrollments to confirm that the changes were made correctly.

SR UNIVERSITY

Task Description- 4: (Execute Aggregate Queries)
Instructions: Use AI tools to write aggregate queries:
• Count number of students enrolled per course
• Display courses with more than one student enrolled
Analyze the correctness of results.
Sample Code:
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
Expected Output:
• Correct count of students per course is displayed.
• Courses with multiple enrollments are listed accurately.

Prompt:-
write a aggregate queries to count numbers of students enrooled per course and dispaly courses with more than one student enrolled and analyze the correctness of results.

Code:-

```
-- Task-04
-- write a aggregate queries to count numbers of students en
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

SELECT c.course_name
FROM course_codes c
JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;

SELECT * FROM courses;
SELECT * FROM enrollments;
```

Output:-

SQL ▾ 1 / 1 1 - 4 of 4

course_id	course_name	course_code
NULL	Mathematics	MATH101
NULL	Physics	PHYS101
NULL	Chemistry	CHEM101
NULL	Biology	BIO101

SQL ▾ 1 / 1 1 - 5 of 5

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

OBSERVATION:

The first query counts how many students are enrolled in each course by joining the *Courses* and *Enrollments* tables and grouping the results by course name. The second query builds on this by showing only those courses that have more than one student, using the HAVING clause. To check if everything is correct, you can compare the results with the actual data in the *Courses* and *Enrollments* tables.

Task Description- 5: (Query Optimization)
Instructions:
• Use AI tools to analyze query performance.
• Rewrite inefficient sub queries using JOINs and test both queries for correctness.
Sample Code:
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101);
-- Optimized Query
SELECT s.*FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
Expected Output:
• Both queries return the same set of students.
• Optimized query performs better on large datasets.

Prompt:_
Analyze the queries performance by rewriting the above queries using different join types (INNER JOIN, LEFT JOIN, RIGHT JOIN) and compare the execution plans to understand the impact of join types on query performance and test both query results.

Code:-

```
-- Task-05
--analyze the queries performance by rewriting the above qu
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;

SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output:-

SQL ▾ 1 / 1 1 - 0 of 0

```
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

SQL ▾ 1 / 1 1 - 3 of 3

student_id	first_name	last_name	email
NULL	John	Doe	john.doe@gmail.com
NULL	Jane	Smith	jane.smith@gmail.com
NULL	Bob	Johnson	bob.johnson@gmail.com

SQL ▾ 1 / 1 1 - 5 of 5

enrollment_id	student_id	course_id	enrollment_date
NULL	1	1	2024-01-15
NULL	1	2	2024-01-16
NULL	2	1	2024-01-17
NULL	2	3	2024-01-18
NULL	3	4	2024-01-19

OBSERVATION:

The optimized query uses an INNER JOIN to directly connect the *Students* and *Enrollments* tables using student_id, making the query simpler and faster. In the original version with a subquery, the database may need to repeatedly check the *Students* table for each result, which can slow things down. With a JOIN, it can quickly match the data, especially if there's an index on student_id. Plus, databases usually handle JOINs more efficiently, so the overall performance improves.